COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S1
DATE PERFORMED:	Jul 28, 2026	**DATE SUBMITTED:**	Aug 3, 2026
NAME:	Jave Joaquin Obias	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 4.1 **[Stacks]**

1. PROCEDURE OUTPUT

ILO A: Create a stack using the C++ STL

SOURCE CODE

```
#include <iostream>
#include <stack> //calling stack from the STL

int main () {
   //create an object for the stack
   std::stack<int> newStack;

   //checking if the stack is empty (empty:true)
   std::cout << "Stack empty? " << newStack.empty() << std::endl;

   //push to the stack
   newStack.push(3);
   newStack.push(8);
   newStack.push(15);

   //checking if the stack is empty (empty:false)
   std::cout << "Stack empty? " << newStack.empty() << std::endl;

   //checking the size of the stack (size:3)
   std::cout << "The size of the stack size: " << newStack.size() <<
std::endl;

   //checking the top of the stack (top:15)
   std::cout << "The top of the element is: " << newStack.top() <<
std::endl;

   //removing the element from the top of the stack
   std::cout << "Removing elements from the stack: \n" ;
   newStack.pop();
   std::cout << "The top of the element is " << newStack.top() <<
std::endl; //checking the top
   newStack.pop();
   std::cout << "The top of the element is " << newStack.top() <<
```

```
std::endl; //checking the top
   newStack.pop();

   //checking if the stack is empty (empty:true)
   std::cout << "Stack empty? " << newStack.empty() << std::endl;

   return 0;
}
```

OUTPUT

```
Stack empty? 1
Stack empty? 0
The size of the stack size: 3
The top of the element is: 15
Removing elements from the stack:
The top of the element is 8
The top of the element is 3
Stack empty? 1
```

ANALYSIS:

Based on my analysis, the program is using the function push() if you want to add an element on the top of the stack. In this program, the values '3', '8', and '15' are added to the stack, and since a stack follows the last in, first out principle, '15' becomes the top element. When using the function pop(), the top element from the stack is removed, so '15' is removed first, followed by '8', and then '3'. The function top() is used to check the current top element, while empty() checks if the stack has no elements and size() checks the number of elements in the stack.

Table 4-1. Output of ILO A

ILO B: Develop C++ code that uses both arrays and linked lists to create a stack

SOURCE CODE

```
#include <iostream>

//static array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;
//always -1 is the top of the stack if it is empty

//functions
void push();
void pop();
void Top();
```

```
bool isEmpty();
bool isFull();
void display();

int main() {
   int choice;
   std::cout << "Enter the number of max elements for the new stack: ";
   std::cin >> i;
   push();

   while(true) {
       std::cout << "\nstack operations:" << std::endl;
       std::cout << "1. PUSH 2. POP 3. TOP 4. ifEMPTY 5. DISPLAY 6. END"
<< std::endl;
       std::cin >> choice;
       switch(choice) {
           case 1: push();
           break;
           case 2: pop();
           break;
           case 3: Top();
           break;
           case 4: std::cout << isEmpty() << std::endl;
           break;
           case 5: display();
           break;
           case 6: std:: cout << "end of the program" << std::endl;
           return 0;
           default: std::cout << "invalid choice." << std::endl;
           break;
       }
   }

   return 0;
}

bool isEmpty() {
   //create a condition to check whether the top element is equal to -1
   if (top == -1) return true;
   return false;
}

bool isFull() {
   //check if we reached the maximum capacity of the array
   if (top >= i-1) return true;
   return false;
}

void push() {
   //check if full -> if yes return an error
   if (isFull()) {
       std::cout << "stack overflow" << std::endl;
       return;
```

```
    }

    std::cout << "\nNew Value to add into the stack:" << std::endl;
    std::cin >> newData;

    //increment the top(index by 1) then assign the new data;
    stack[++top] = newData;
}

void Top() {
    //check if the stack is empty
    if (isEmpty()) {
        std::cout << "stack underflow" << std::endl;
    }

    //
    std::cout << "The element on the top of the stack is " << stack[top] <<
std::endl;
}

void pop() {
    //check if the stack is empty
    if (isEmpty()) {
        std::cout << "stack underflow" << std::endl;
    }

    //display the top value
    std::cout << "popping" << stack[top] << std::endl;
    top--;
}

void display() {
    //printing all the elements on the stack
    std:: cout << "STACK:" << std::endl;
    for (int j = top; j != -1; j--) {
        std::cout << stack[j] << std::endl;
    }
}
```

OUTPUT

```
Enter the number of max elements for the new stack: 3

New Value to add into the stack:
3

stack operations:
1. PUSH 2. POP 3. TOP 4. ifEMPTY 5. DISPLAY 6. END
1

New Value to add into the stack:
2

stack operations:
1. PUSH 2. POP 3. TOP 4. ifEMPTY 5. DISPLAY 6. END
1

New Value to add into the stack:
1

stack operations:
1. PUSH 2. POP 3. TOP 4. ifEMPTY 5. DISPLAY 6. END
3
The element on the top of the stack is 1
```

```
stack operations:
1. PUSH 2. POP 3. TOP 4. ifEMPTY 5. DISPLAY 6. END
2
popping1

stack operations:
1. PUSH 2. POP 3. TOP 4. ifEMPTY 5. DISPLAY 6. END
3
The element on the top of the stack is 2

stack operations:
1. PUSH 2. POP 3. TOP 4. ifEMPTY 5. DISPLAY 6. END
5
STACK:
2
3

stack operations:
1. PUSH 2. POP 3. TOP 4. ifEMPTY 5. DISPLAY 6. END
6
end of the program

Process finished with exit code 0
```

ANALYSIS:

Based on my analysis, this program demonstrates how to create a stack using a static array. The push() function is used to add a new element to the top of the stack, while the pop() function removes the top element. The Top() function is used to display the current top element without removing it. The program also uses isEmpty() to check if the stack has no elements and isFull() to check if the stack has reached its maximum capacity. The variable top starts at '-1', which means that the stack is initially empty, and it increases whenever a new element is added and decreases whenever an element is removed. The display() function shows all the elements in the stack starting from the top.

Table 4-2. Output of ILO B.1

SOURCE CODE

```cpp
#include <iostream>
#include "CPE010_HOA4_mainB2.h"

int main() {
   std::cout << "--- PUSHING 'C' 'P' 'E' ONTO THE STACK --- \n";

   //push
   push('C');
   push('P');
   push('E');

   //display the current top
   Top<char>(); //Top should be 'e'

   std::cout << "\n--- DISPLAY ALL --- \n";
   displayAll<char>();

   std::cout << "--- POPPING ALL THE ELEMENTS --- \n";

   //pop
   while (!isEmpty<char>()) {
       std::cout << "Popped: " << pop<char>() << std::endl;
   }

   std::cout << "\n--- TESTING THE UNDERFLOW --- \n";
   pop<char>();

   return 0;
}
```

```cpp
#ifndef OBIAS_CPE010_HOA4_MAINB2_H
#define OBIAS_CPE010_HOA4_MAINB2_H
#include <iostream>
#include <ostream>

//class node that contains data and the next
template <typename T>
class Node {
   public:
   T data;     //value that we need to store
   Node *next; //pointer to the next node
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr; //pointer

//operations for the stack
```

```cpp
//push
template <typename T>
void push(T newData) {
    //create a new node and assign the newData
    Node<T> *newNode = new Node<T>;
    newNode->data = newData;

    //new node points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}

//pop
template <typename T>
T pop() {
    //check if the stack is empty
    if (head<T> == nullptr) {
        std::cout << "Stack Overflow" << std::endl;
    }

    //save the current top node
    Node<T> *temp = head<T>;

    //store its data
    T tempVal = temp->data;

    //move the head to the next node
    head<T> = head<T>->next;

    //free the removed node
    delete temp;

    //return the removed value
    return tempVal;
}

//isEmpty
template <typename T>
bool isEmpty() {
    return head<T> == nullptr;
}

//top
template <typename T>
void Top() {
    //error catching:
    if (head<T> == nullptr) {
        std::cout << "Stack Underflow" << std::endl;
        return;
    }
```

```
    //display the top node
    std::cout << "Top of Stack: "<< head<T>->data << std::endl;
}

//displayAll
template <typename T>
void displayAll() {
    if (head<T> == nullptr) {
        std::cout << "Stack Underflow." << std::endl;
        return;
    }

    std::cout << "Stack: " << std::endl;
    Node<T> *temp = head<T>;
    while (temp != nullptr) {
        std::cout << temp->data << std::endl;
        temp = temp->next;
    }

    std::cout << std::endl;
}

#endif
```

OUTPUT

```
--- PUSHING 'C' 'P' 'E' ONTO THE STACK ---
Top of Stack: E

--- DISPLAY ALL ---
Stack:
E
P
C

--- POPPING ALL THE ELEMENTS ---
Popped: E
Popped: P
Popped: C

--- TESTING THE UNDERFLOW ---
Stack Overflow
```

ANALYSIS:
Based on my analysis, this program demonstrates how to use a stack with template functions to store and manage character elements. The push() function adds 'C', 'P', and 'E' to the stack, following the last in, first out principle, so 'E' becomes the top element. The Top<char>() function checks the current top element, while displayAll<char>() displays all the elements in the stack. The pop<char>() function then removes each element starting from 'E', followed by 'P', and finally 'C'. The isEmpty<char>() function checks if the stack still contains elements, and after all elements are removed, the program tests underflow by calling pop<char>() again when the stack is already empty.

Table 4-3. Output of ILO B.2

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack:

SOURCE CODE

```
#include <iostream>

// Stack using Array
const int maxcap = 100;
```

```
char stack[maxcap];
int top = -1;
std::string exp;

// functions
void push(char newdata);
char pop();
bool isEmpty();
bool isFull();
bool isBalanced();

int main() {

   char c;

   while (c != 'n') {

       std::cout << "Enter the Expression: ";
       std::cin >> exp;

       if (isBalanced()) {
           std::cout << "Expression is VALID." << std::endl;
       }
       else {
           std::cout << "Expression is INVALID." << std::endl;
       }

       std::cout << "Continue? (y/n): ";
       std::cin >> c;
       std::cout << std::endl;
   }

   return 0;
}

bool isBalanced() {
   char temp;
   bool valid = true;

   // reset the stack
   top = -1;

   for (int i = 0; i < exp.length(); i++) {

       // check if it is opening bracket
       if (exp[i] == '(' || exp[i] == '[' || exp[i] == '{') {
           push(exp[i]);
       }

       // check if it is closing bracket
       else if (exp[i] == ')' || exp[i] == ']' || exp[i] == '}') {

           // check if stack is empty
```

```
            if (isEmpty()) {
                valid = false;
                break;
            }

            // get the top bracket
            temp = pop();

            // compare opening and closing brackets
            if (exp[i] == ')' && temp != '(') {
                valid = false;
                break;
            }

            if (exp[i] == ']' && temp != '[') {
                valid = false;
                break;
            }

            if (exp[i] == '}' && temp != '{') {
                valid = false;
                break;
            }
        }
    }

    // check if there are still brackets left
    if (!isEmpty()) {
        valid = false;
    }

    return valid;
}

void push(char newdata) {
    // check if stack is full
    if (isFull()) {
        std::cout << "Stack Overflow" << std::endl;
        return;
    }

    // increment the top of stack,then add newdata
    stack[++top] = newdata;
}

char pop() {
    // check if stack is empty
    if (isEmpty()) {
        return ' ';
    }

    // get the top value then decrement top
    char temp = stack[top];
```

```
    top--;

    return temp;
}

bool isEmpty() {
    // check if stack is empty
    if (top == -1)
        return true;

    return false;
}

bool isFull() {
    // check if stack is full
    if (top == maxcap - 1)
        return true;

    return false;
}
```

OUTPUT

```
Enter the Expression: (A+B)+(C-D)
Expression is VALID.
Continue? (y/n): y

Enter the Expression: ((A+B)+(C-D)
Expression is INVALID.
Continue? (y/n): y

Enter the Expression: ((A+B)+[C-D])
Expression is VALID.
Continue? (y/n): y

Enter the Expression: ((A+B]+[C-D]}
Expression is INVALID.
Continue? (y/n): n
```

Table 4-4. Output of ILO C/Supplementary

CONCLUSION
In this activity, I learned how to program a stack by using the functions push, pop, top, isEmpty, isFull. I use a stack to check if an expression is balanced. It pushes every opening symbol into the stack and removes it when the correct closing symbol is found. I created a function that will check if all the symbols match and the stack is empty at the end, the expression will be valid. Also, the program displays if the expression will be invalid because there are missing or mismatched symbols.

Table 4-5. Conclusion

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.

I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.